For these reasons, it’s beneficial to use an alternative authentication mechanism. One alternative supported by many authorization servers is the JWT Bearer grant type for OAuth2, defined in RFC 7523 (https://tools.ietf.org/html/rfc7523). This specification allows a client to obtain an access token by presenting a JWT signed by a trusted party, either to authenticate itself for the client credentials grant, or to exchange a JWT representing authorization from a user or service account. In the first case the JWT is signed by the client itself using a key that it controls. In the second case, the JWT is signed by some authority that is trusted by the AS, such as an external OIDC provider. This can be useful if the AS wants to delegate user authentication and consent to a 3rd party service. For service account authentication, the client is often directly trusted with the keys to sign JWTs on behalf of that service account because there is a dedicated service account for each client. In section 11.5.3 you’ll see how separating the duties of the client from the service account authentication can add an extra layer of security.

By using a public key signature algorithm, the client needs to supply only the public key to the AS, reducing the risk if the AS is ever compromised because the public key can only be used to verify signatures not create them. Adding a short expiry time also reduces the risks when authenticating over an insecure channel, and some servers support remembering previously used JWT IDs to prevent replay.

Another advantage of JWT bearer authentication is that many authorization servers support fetching the client’s public keys in JWK format from a HTTPS endpoint. The AS